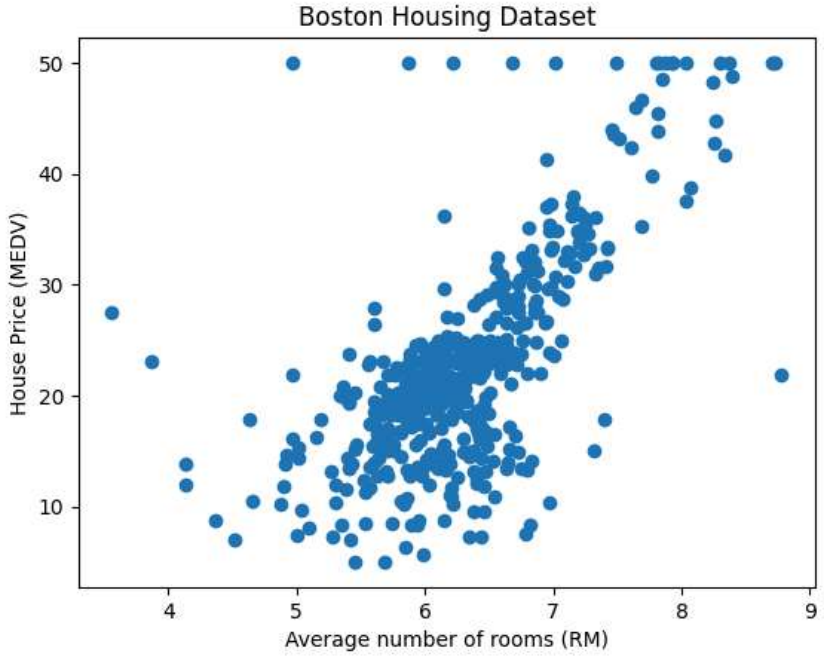
1)Load Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data_url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.data"
columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV']

# Read the data, replacing delim_whitespace with sep='\s+'
housing = pd.read_csv(data_url, sep='\s+', names=columns)

# Display the head of the dataframe
display(housing.head())

# Select 'RM' as the feature for the scatter plot

X = housing['RM'].values.reshape(-1, 1) # shape (m, 1)
y = housing['MEDV'].values.reshape(-1, 1) # shape (m, 1); MEDV is in $1000s




# Create the scatter plot
plt.scatter(X, y)
plt.xlabel("Average number of rooms (RM)")
plt.ylabel("House Price (MEDV)")
plt.title("Boston Housing Dataset")
plt.show()
```

```
<>:9: SyntaxWarning: invalid escape sequence '\s'
<>:9: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-3203547483.py:9: SyntaxWarning: invalid escape sequence '\s'
  housing = pd.read_csv(data_url, sep='\s+', names=columns)
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2

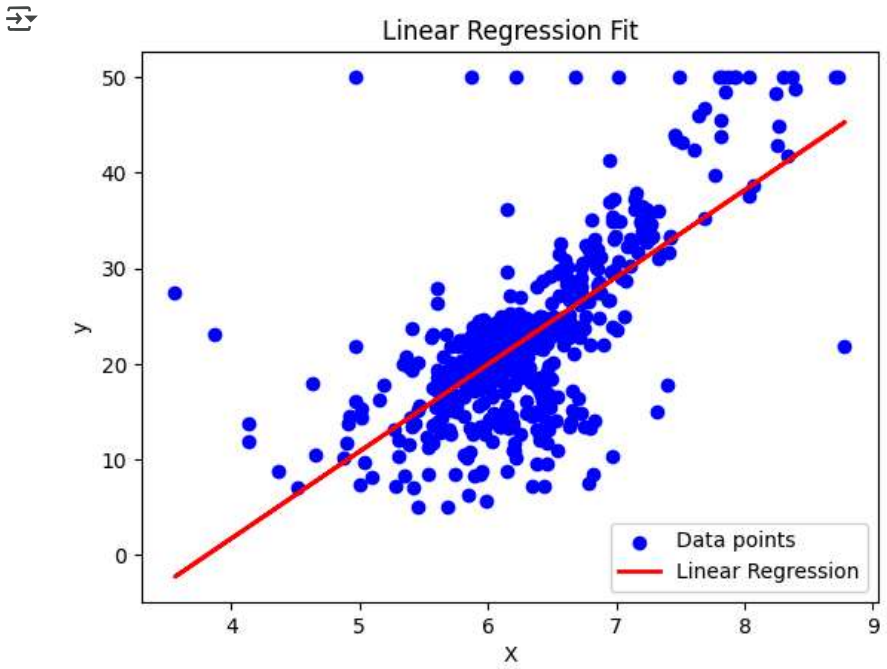


2)Split the Dataset

```
from sklearn.model_selection import train_test_split
X=housing.drop("MEDV",axis=1)
y=housing["MEDV"]

X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

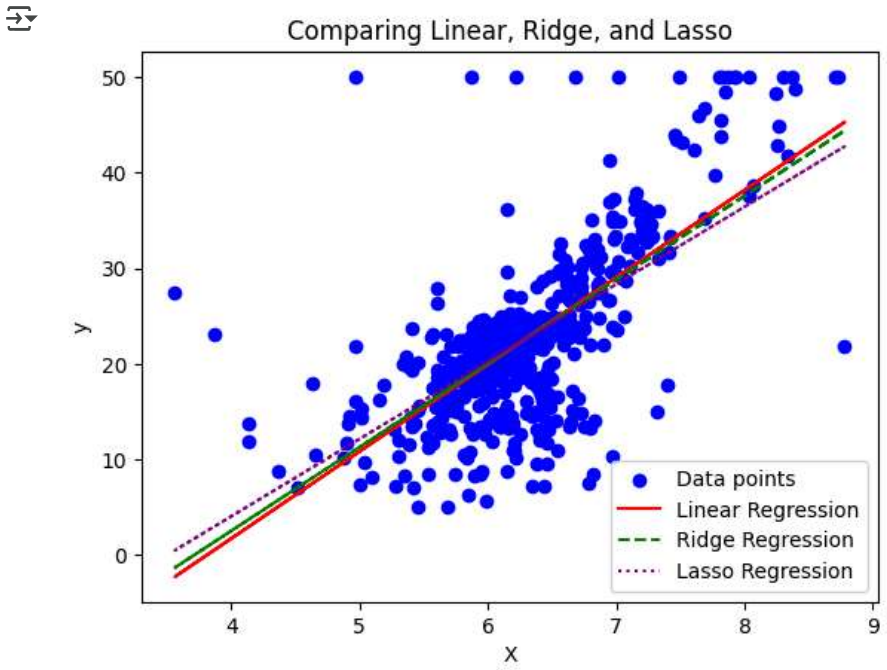
3)Train models

```
from sklearn.linear_model import LinearRegression
# Train Linear Regression
lin_reg = LinearRegression()
# Use the single feature 'RM' for training and plotting for visualization purposes
X = housing['RM'].values.reshape(-1, 1)
y = housing['MEDV'].values.reshape(-1, 1)
lin_reg.fit(X, y)  # fit model to data
y_pred = lin_reg.predict(X)  # predictions on training data

# Plot regression line
plt.scatter(X, y, color='blue', label='Data points')
plt.plot(X, y_pred, color='red', linewidth=2, label='Linear Regression')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Linear Regression Fit')
plt.legend()
plt.show()
```




```
# Train Ridge and Lasso
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge,Lasso
ridge = Ridge(alpha=10)   # alpha = regularization strength
lasso = Lasso(alpha=0.5)
```

```
# Use the single feature 'RM' for training and plotting for visualization purposes
X_ = housing['RM'].values.reshape(-1, 1)
y_single_feature = housing['MEDV'].values.reshape(-1, 1)

ridge.fit(X_, y)
lasso.fit(X, y)

y_ridge = ridge.predict(X)
y_lasso = lasso.predict(X)

# Compare models
plt.scatter(X, y_single_feature, color='blue', label='Data points')
plt.plot(X, y_pred, color='red', label='Linear Regression')
plt.plot(X, y_ridge, color='green', linestyle='--', label='Ridge Regression')
plt.plot(X, y_lasso, color='purple', linestyle=':', label='Lasso Regression')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Comparing Linear, Ridge, and Lasso')
plt.legend()
plt.show()
```



Double-click (or enter) to edit

4)Evaluation of Models

```
from sklearn.metrics import mean_squared_error
mse=mean_squared_error(y,y_pred)
print("MSE",mse)
```

```
MSE 43.60055177116956
```

```
from sklearn.metrics import mean_absolute_error
mae=mean_absolute_error(y,y_pred)
print("MAE",mae)
```

```
MAE 4.4477729015322325
```

```
from sklearn.metrics import root_mean_squared_error
rmse=np.sqrt(mean_squared_error(y,y_pred))
print("RMSE",rmse)
```

```
RMSE 6.603071389222561
```

```
from sklearn.metrics import r2_score
r2_score=r2_score(y,y_pred)
print("R2",r2_score)
```

```
R2 0.48352545599133423
```

5)Compare Results

```
from sklearn.linear_model import Ridge,Lasso
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Lasso Regression": Lasso(alpha=0.1)
}

# Evaluate models
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2_value = r2_score(y_test, y_pred)
```

```
    mae = mean_absolute_error(y_test, y_pred)

    results.append({
        "Model": name,
        "MSE": round(mse, 2),
        "RMSE": round(rmse, 2),
        "R²": round(r2_value, 2),
        "MAE": round(mae, 2)
    })

# Create comparison table
df_results = pd.DataFrame(results)
df_results
```

	Model	MSE	RMSE	R²	MAE
0	Linear Regression	46.14	6.79	0.37	4.48
1	Ridge Regression	46.09	6.79	0.37	4.48
2	Lasso Regression	45.92	6.78	0.37	4.47

```
Start coding or generate with AI.
```